EXPERIMENT 8: Two-Layer Neural Network with Backpropagation

1. Aim

To implement a two-layer neural network from scratch using NumPy, perform forward propagation and backpropagation using sigmoid activation, train the network using gradient descent, plot the loss curve and decision boundary, and verify gradients numerically.

2. Mathematical Formulae

Forward propagation

$$Z^{[1]} = XW^{[1]} + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = A^{[1]}W^{[2]} + b^{[2]}$$

$$\hat{Y} = \sigma(Z^{[2]})$$

Binary cross-entropy loss

$$J = -\frac{1}{m}\sum_{i=1}^{m}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

Backpropagation for the output layer

$$dZ^{[2]} = \hat{Y} - Y$$

$$dW^{[2]} = \frac{(A^{[1]})^T dZ^{[2]}}{m}$$

Backpropagation for the hidden layer

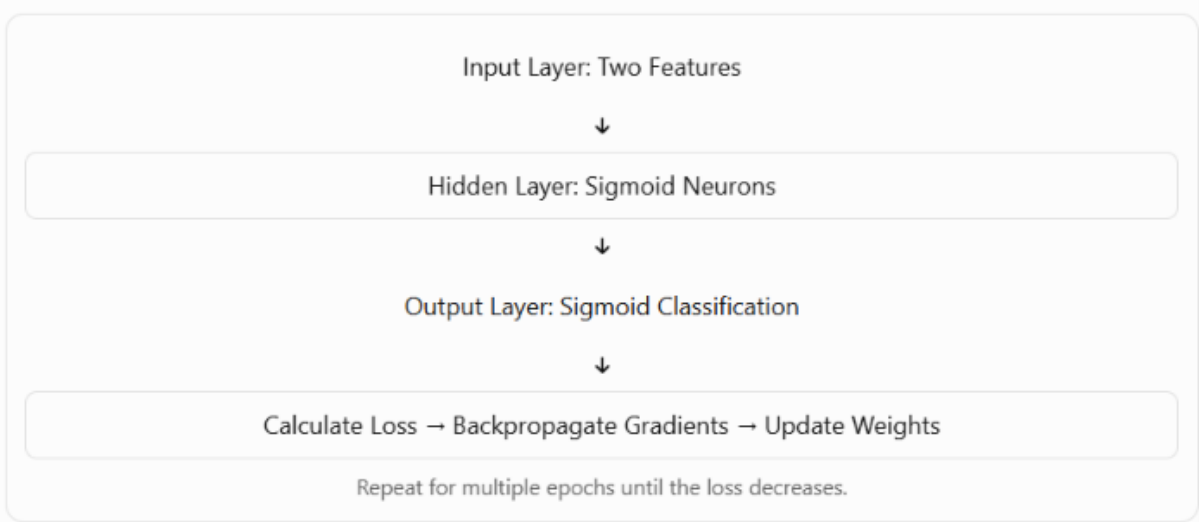
$$dZ^{[1]} = (dZ^{[2]}(W^{[2]})^T) \odot A^{[1]}(1 - A^{[1]})$$

Gradient descent update

$$W^{new} = W^{old} - \eta dW$$

Where m is the number of samples and η is the learning rate.

3. System Architecture



4. Algorithm / Procedure

1. Generate an XOR dataset.
2. Initialize weights randomly and biases to zero.
3. Perform forward propagation through the hidden and output layers.
4. Calculate binary cross-entropy loss.
5. Calculate gradients using the chain rule.
6. Update weights and biases using gradient descent.
7. Plot loss against epochs.
8. Plot the learned decision boundary.
9. Verify one weight gradient using numerical differentiation.

5. Python Program

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# XOR dataset
X = np.array([
  [0, 0],
  [0, 1],
```

```
    [1, 0],
    [1, 1]
], dtype=float)

Y = np.array([[0], [1], [1], [0]], dtype=float)

# Network dimensions
input_size = 2
hidden_size = 4
output_size = 1

W1 = np.random.randn(input_size, hidden_size) * 0.8
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size) * 0.8
b2 = np.zeros((1, output_size))

def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))

def forward(X):
    Z1 = X @ W1 + b1
    A1 = sigmoid(Z1)

    Z2 = A1 @ W2 + b2
    A2 = sigmoid(Z2)

    return A1, A2

def binary_cross_entropy(Y, predictions):
    eps = 1e-8
```

```
    predictions = np.clip(predictions, eps, 1 - eps)

    return -np.mean(
        Y * np.log(predictions)
        + (1 - Y) * np.log(1 - predictions)
    )

learning_rate = 1.0
epochs = 10000
losses = []

for epoch in range(epochs):
    # Forward propagation
    A1, A2 = forward(X)

    # Loss
    loss = binary_cross_entropy(Y, A2)
    losses.append(loss)

    # Backpropagation
    m = len(X)

    dZ2 = A2 - Y
    dW2 = A1.T @ dZ2 / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * A1 * (1 - A1)
    dW1 = X.T @ dZ1 / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m
```

```
    # Gradient descent
    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2
    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

# Final predictions
A1, predictions = forward(X)
classes = (predictions >= 0.5).astype(int)

print("Actual XOR labels:", Y.ravel())
print("Predicted probabilities:", np.round(predictions.ravel(), 3))
print("Predicted classes:", classes.ravel())
print("Final loss:", round(losses[-1], 6))

# Loss curve
plt.figure(figsize=(7, 5))
plt.plot(losses)
plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Backpropagation Training Loss")
plt.show()

# Decision boundary
grid_x, grid_y = np.meshgrid(
    np.linspace(-0.5, 1.5, 200),
    np.linspace(-0.5, 1.5, 200)
)

grid = np.c_[grid_x.ravel(), grid_y.ravel()]
_, grid_predictions = forward(grid)
```

```
grid_predictions = grid_predictions.reshape(grid_x.shape)


plt.figure(figsize=(7, 5))
plt.contourf(grid_x, grid_y, grid_predictions, levels=20)
plt.scatter(
   X[:, 0], X[:, 1],
   c=Y.ravel(), edgecolors="black", s=100
)
plt.xlabel("Input 1")
plt.ylabel("Input 2")
plt.title("Two-Layer Neural Network Decision Boundary")
plt.show()


# Numerical gradient verification for W2[0, 0]
A1, A2 = forward(X)
dZ2 = A2 - Y
dW2 = A1.T @ dZ2 / len(X)
analytic_gradient = dW2[0, 0]


epsilon = 1e-5
original = W2[0, 0]


W2[0, 0] = original + epsilon
_, plus_predictions = forward(X)
loss_plus = binary_cross_entropy(Y, plus_predictions)


W2[0, 0] = original - epsilon
_, minus_predictions = forward(X)
loss_minus = binary_cross_entropy(Y, minus_predictions)


W2[0, 0] = original
```

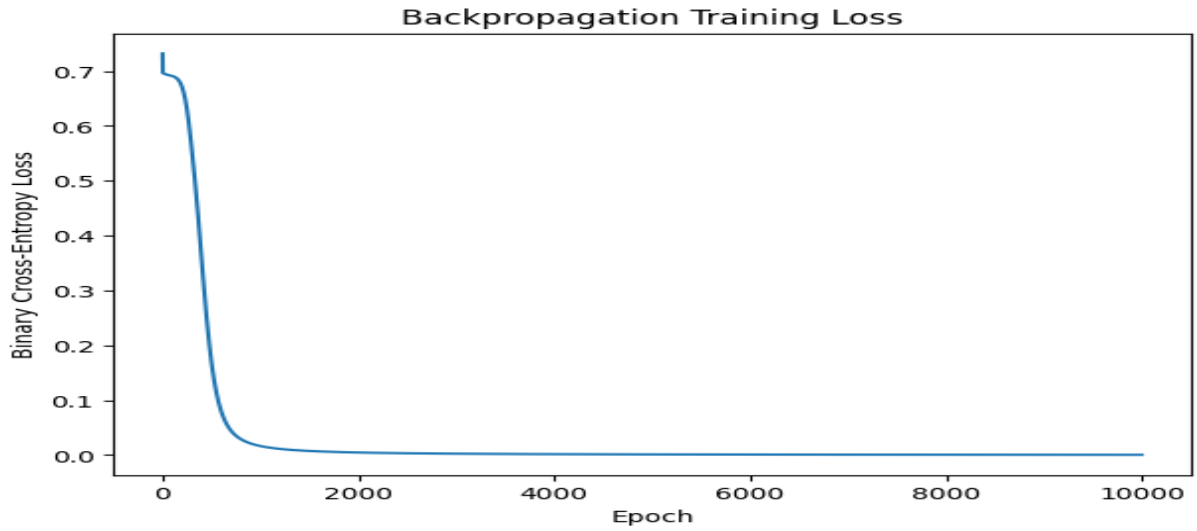
numeric_gradient = (loss_plus - loss_minus) / (2 * epsilon)

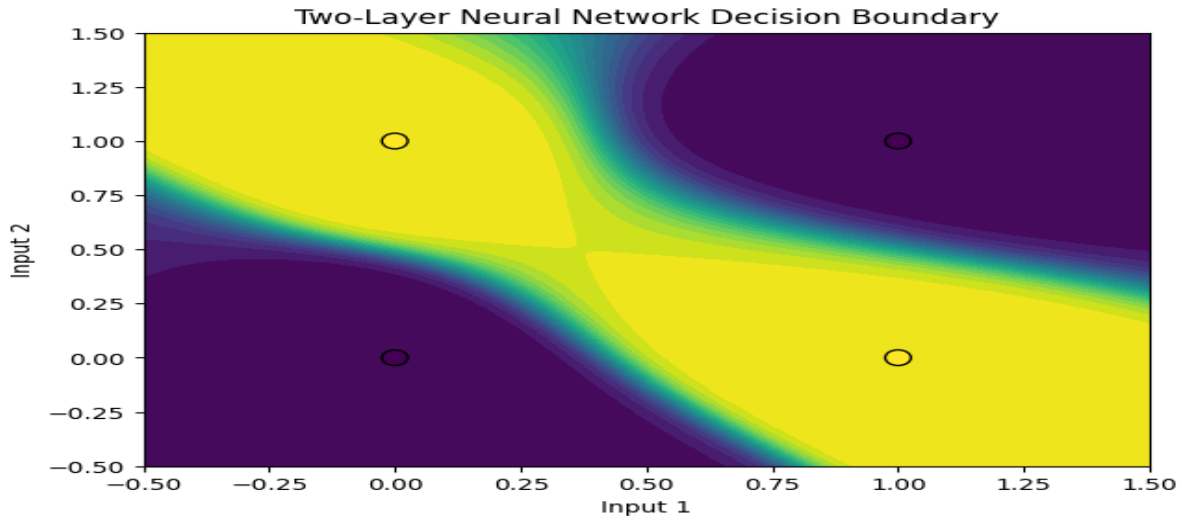
print("Analytical gradient:", analytic_gradient)

print("Numerical gradient:", numeric_gradient)

6. Output

```
Actual XOR labels: [0. 1. 1. 0.]
Predicted probabilities: [0.    1.    0.999 0.001]
Predicted classes: [0 1 1 0]
Final loss: 0.000525
```






```
Analytical gradient: 6.28337489834691e-05
Numerical gradient: 6.283374634941281e-05
```

7. Result

A two-layer neural network was implemented from scratch. Forward propagation, binary cross-entropy loss, backpropagation and gradient descent were performed. The model learned a non-linear decision boundary, and numerical differentiation was used to verify the calculated gradient.